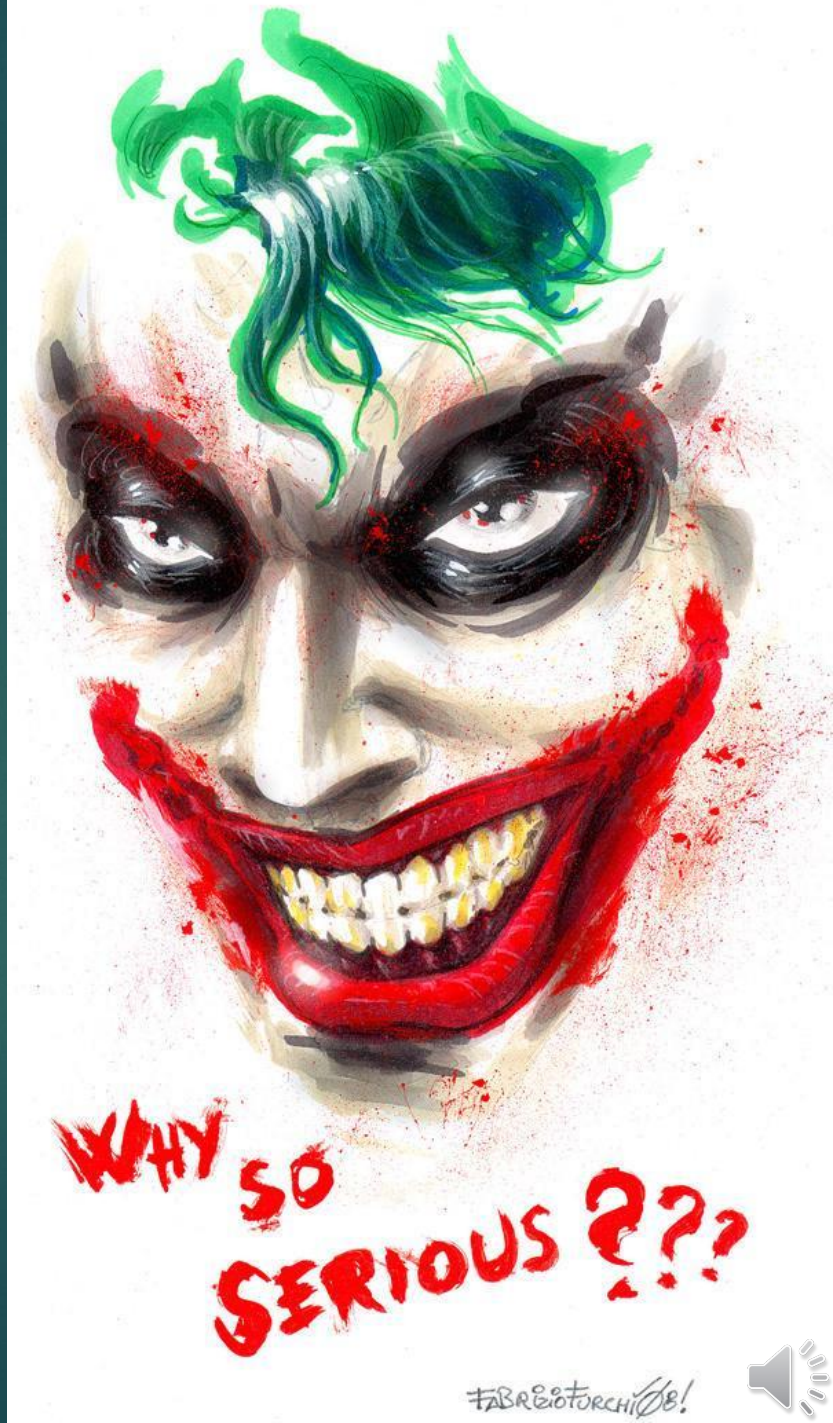


WELCOME TO DAX



DAX overview

Data Analysis Expressions (DAX) is a formula expression language used in Analysis Services, Power BI, and Power Pivot in Excel.

DAX formulas include functions, operators, and values to perform advanced calculations and queries on data in related tables and columns in tabular data models.



What is DAX?

- ▶ DAX helps us to return values after making calculations from the already available data
- ▶ To understand DAX, you just need to be similar with Microsoft Excel formulas

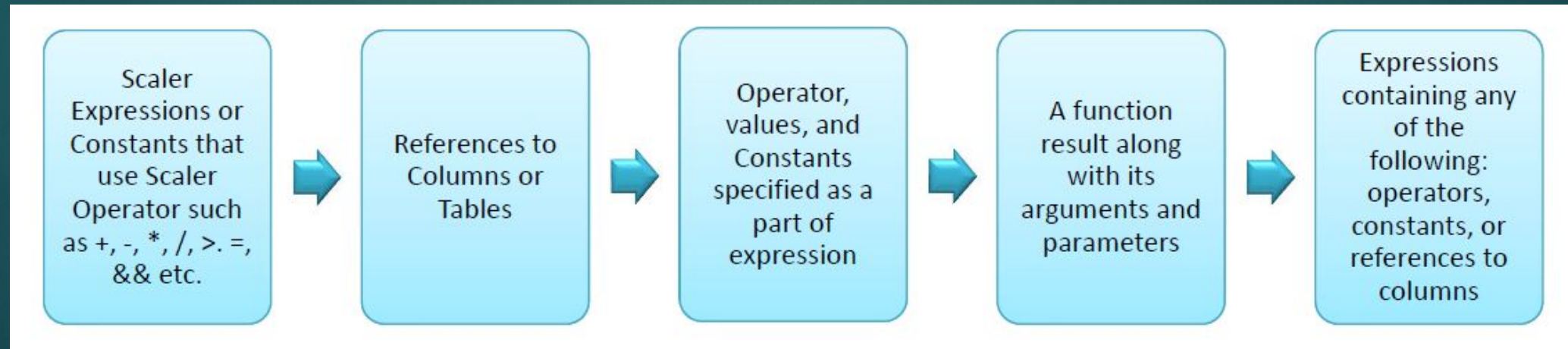


What is DAX?

- ▶ • DAX formulas are just like the ones we issue in Microsoft excel
- ▶ • However, DAX functions and excel functions differ in certain aspects
- ▶ • Excel allows its users to reference cells or arrays. If the users need to perform such behaviour in Power Bi, they would require the use of DAX Functions
- ▶ • DAX provides more data types than Microsoft Excel does

Syntax

- ▶ • DAX formulas begin with an = sign after which any scalar value can be provided
- ▶ The scalar value can be an expression that evaluates to a scalar or an expression that can be converted to a scalar



Syntax

- ▶ • DAX requires that all its objects whether tables or columns must have unique names
- ▶ Also, names of objects are case insensitive, i.e. Products and PRODUCTS would refer to the same table or column
- ▶ A column name should always be fully qualified, i.e. it must be preceded by the table name and should be written in square brackets, e.g. SALES.[Product_Id]
- ▶ Sometimes table names will contain spaces in which case they must be enclosed in single quotations

Syntax

- ▶ • A fully qualified name is required in the following circumstances:

1.

When the VALUES function requires arguments

2.

As arguments to the ALL or ALLEXCEPT functions

3.

While using the CALCULATE or CALCULATE functions passes as a filter argument

4.

As an argument to RELATEDTABLE function

5.

As an argument at any time intelligence function

Functions

- ▶ Functions in DAX can be categorised into the following:

Date and Time Functions

1

2

Filter Functions

Time Intelligence Functions

3

4

Information Functions

Logical Functions

5

6

Math and Trig Functions

Text Functions

7

8

Many others as well



Functions

- ▶ • DAX functions always point to either a column or a table
- ▶ • To specify only selected values you will need to filter them
- ▶ • DAX is also capable of returning a whole table rather than a column only
- ▶ • DAX works with Time Intelligence functions to perform dynamic calculations

Row Context

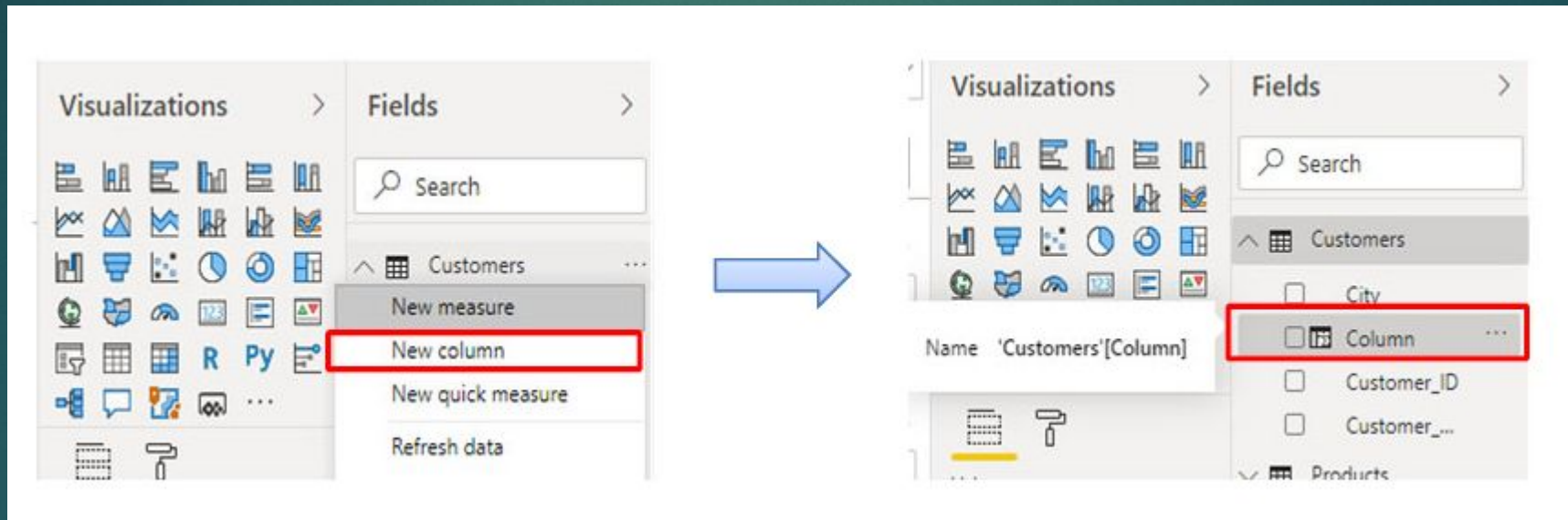
- ▶ • Row context is much easier to understand as compared to filter context
- ▶ • The simplest way to visualise row context is to take a table and add a calculated column
- ▶ • Each row in a table contains its own row context
- ▶ • For instance, if a table has two columns a and b and row 1 values are 1 and 2 respectively
- ▶ • Similarly, row 2 values are 3 and 4 respectively
- ▶ • If you add a column c that sums the value of columns a and b, then the column c value for row 1 would be 3, and the value for row 2 would be seven

Calculated Columns

- With calculated columns, you can append new data to the existing table in your table
- You can create a data analysis expressions (DAX) formula that defines the columns values rather of querying and loading values in your new column from a data source
- In Power BI desktop, calculated columns are generated by using the new column feature in **Report** view
- Calculated columns that you create appear in the fields list just like any other field

Calculated Columns

- But they will contain a special icon showing its values are the result of a formula:



You can name new columns whatever you want, and add them to a report visualisation just like other fields

Calculated Tables

- You can create a calculated table by using the **New Table** feature in a data view or report view of Power BI desktop
- For instance, suppose you are a personnel manager who has a table of **Sales_2019** and another table of **Sales_2020**, and you want to combine both tables into a single table called **Sales**

Sales_2019

Year	▼	Quarter	▼	SalesAmount	▼
2019		1		355000	
2019		2		330000	
2019		3		112000	
2019		4		122000	

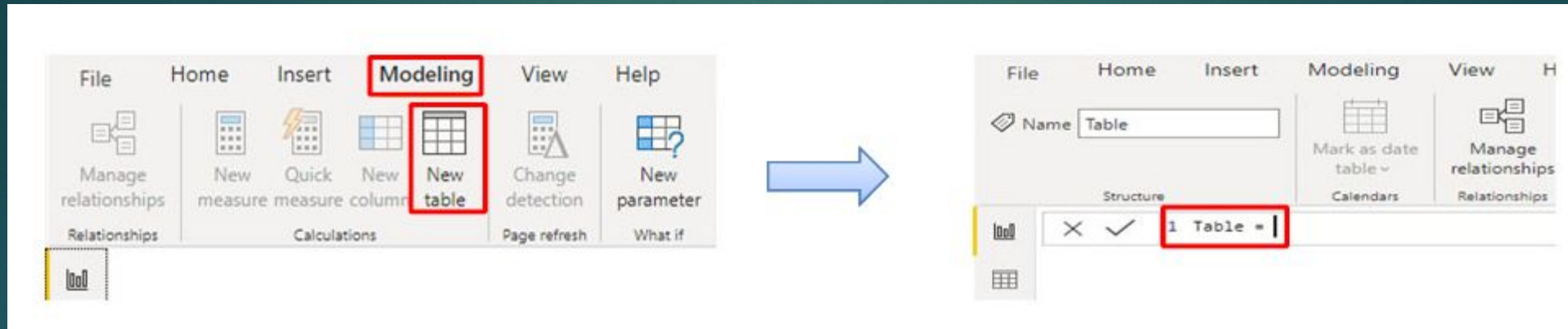
Sales_2020

Year	▼	Quarte	▼	SalesAmount	▼
2020		5		12300	
2020		6		124000	
2020		7		134000	
2020		8		153000	

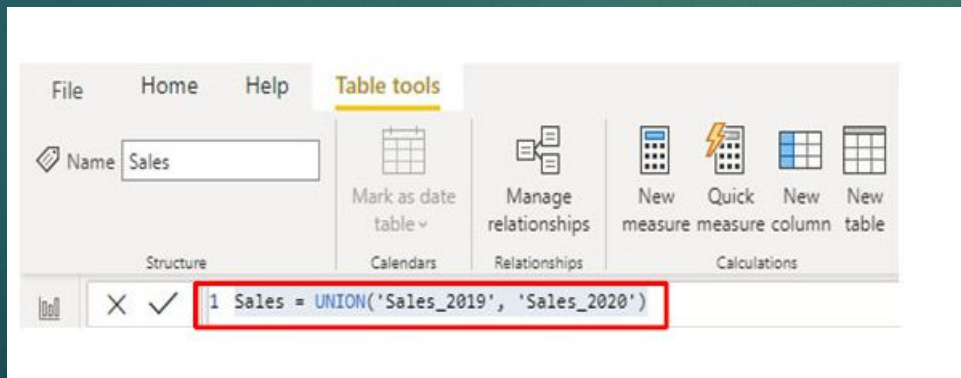
Calculated Tables

- The following are the steps to create a calculated table:

Step 1: Click on the **Modeling Tab** and then select **New Table**



Step 2: Enter the following formula in the formula bar



Calculated Tables

Step 3: A new table named **Sales** is created and appears just like any other table in the **Fields** pane:

The screenshot illustrates the creation of a calculated table in Power BI. On the left, the DAX formula bar shows the formula: `1 Sales = UNION('Sales_2019', 'Sales_2020')`. Below the formula, a table preview is displayed with the following data:

Year	Quarter	SalesAmount
2019	1	355000
2019	2	330000
2019	3	112000
2019	4	122000
2020	1	125000
2020	2	330000
2020	3	112000
2020	4	752000

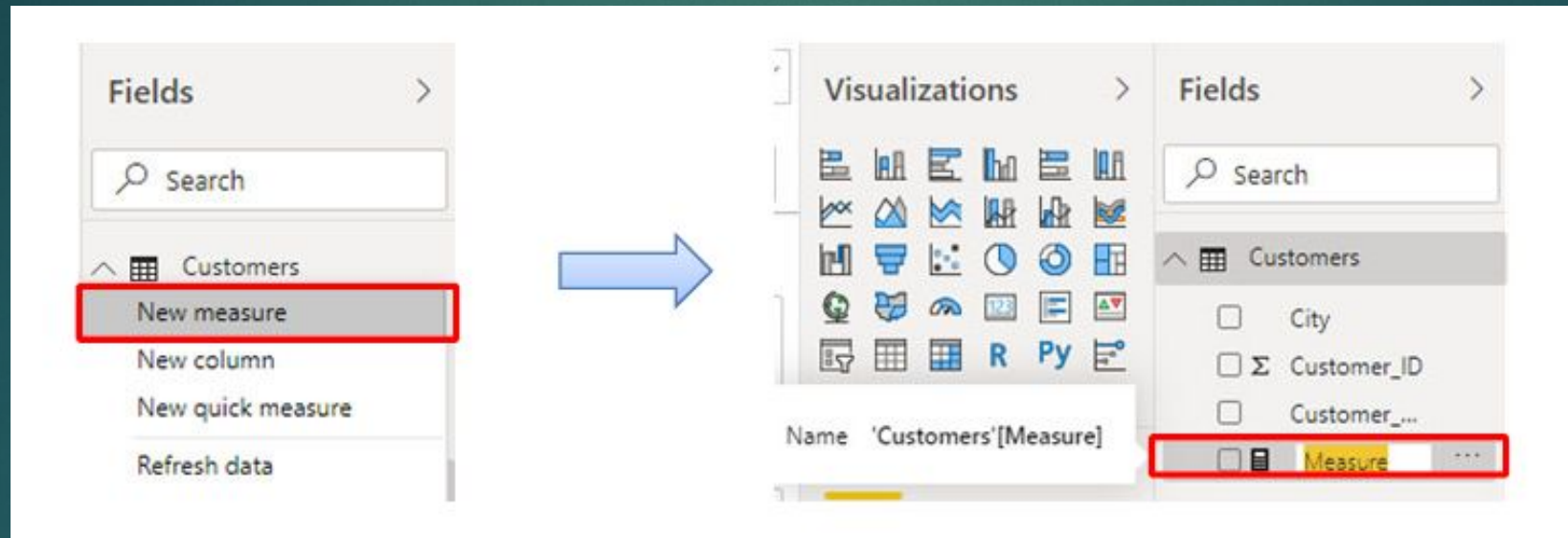
A blue arrow points from the table preview to the Fields pane on the right. The Fields pane shows the 'Sales' table, which is highlighted with a red box. Below 'Sales', the fields 'Quarter', 'SalesAmount', and 'Year' are listed, along with the source tables 'Sales_2019' and 'Sales_2020'.

Measures

- Measures are generally used for data analyses
- Simple summarisations such as sums, averages, counts and minimum, maximum can be set through the **Fields** well
- The calculated results of measures are always varying according to your interaction with your reports, allowing for fast and dynamic ad-hoc data exploration
- In the Power BI desktop, measures are created in a data view or report view
- The measures that you are created appear in the **Fields** list with a calculator icon

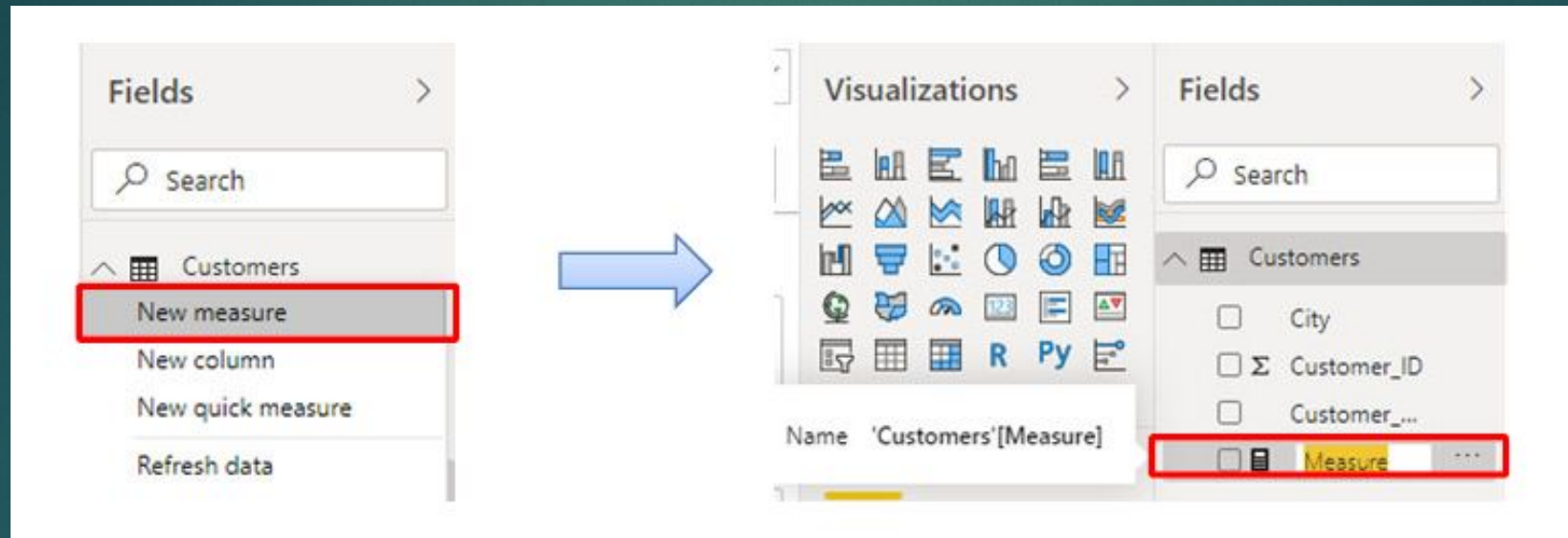
Measures

You can enter the name for measures whatever you want and add them to a new or existing visualisation just like any other field



Measures

You can enter the name for measures whatever you want and add them to a new or existing visualisation just like any other field



What are the Relationships ?

In Power BI, a relationship is used to describe the connections or the relation between two or more tables

- The relationship is used to perform an analysis based on multiple tables
- The relationship helps to display the data as well as correct information between multiple tables
- The relationship is also used to calculate the accurate results



Viewing Relationships ?

The model view displays all of the tables, columns, and relationships in your model

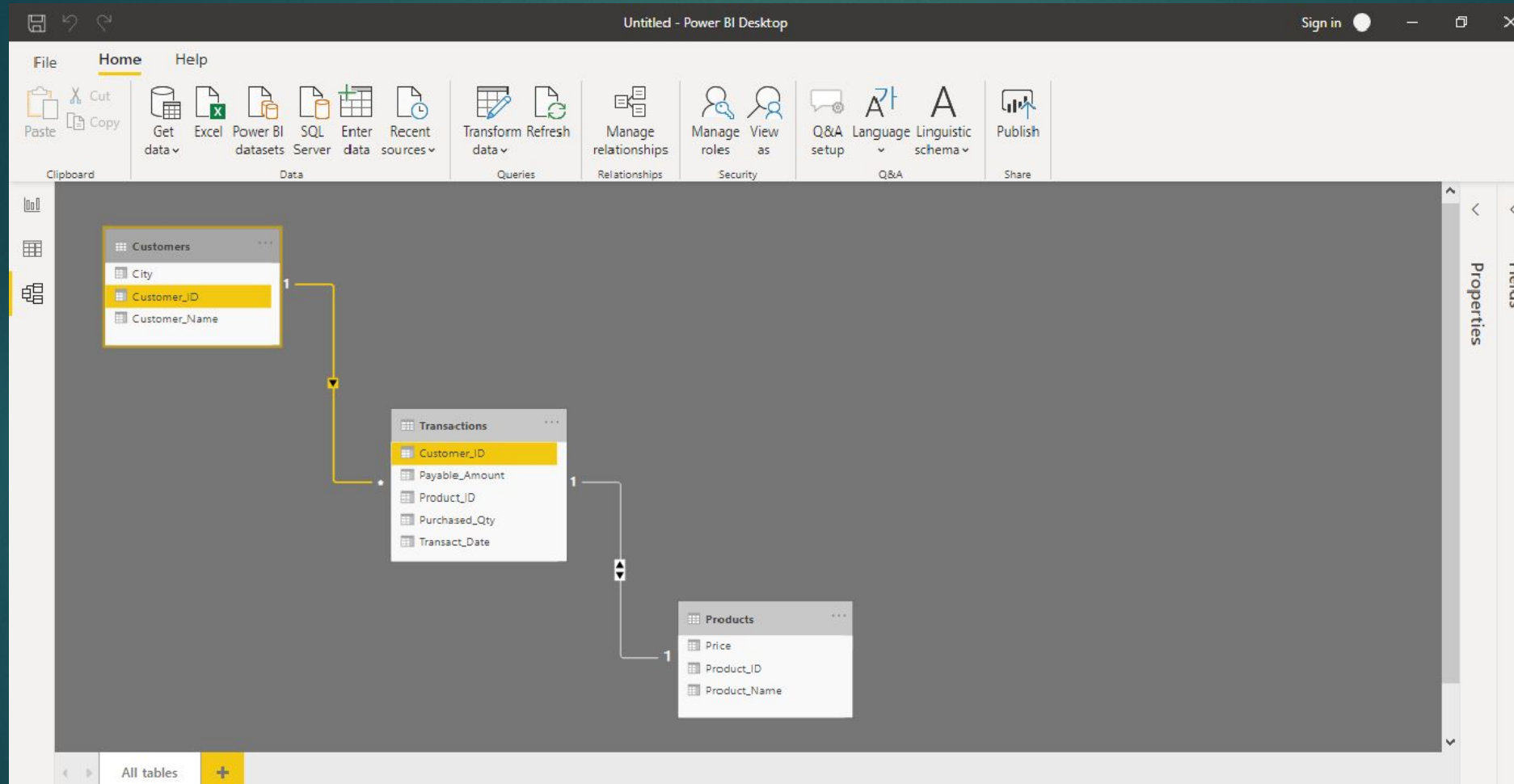
This view can be mainly useful when your model contains complex relationships between many tables

Click on the **Model** icon placed at the left side of the window to see a view of the existing model

Your cursor over a relationship line represent the columns that are used as shown in the next slide:



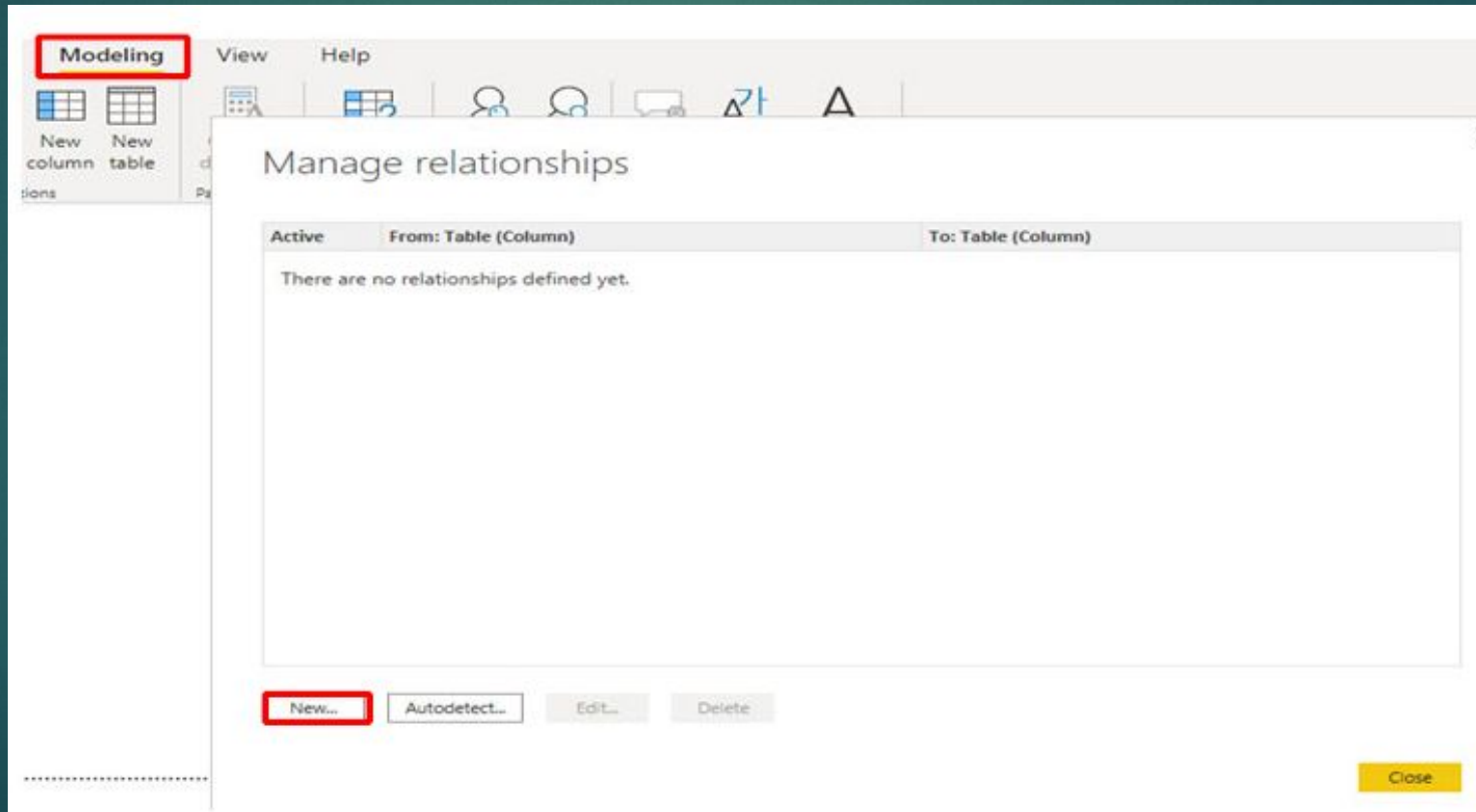
Viewing Relationships ?



Creating Relationships

- The following are the steps to create a relationship manually:

Step 1: On the **Modeling** tab, select **Manage Relationships** > **New**



Creating Relationships

Step 2: In the **Create relationship** dialog box, select a **Products** table in the first table dropdown list, and then choose the column you want to use in the relationship

Create relationship

Select tables and columns that are related.

Customers
Products
Transactions

Cardinality

Cross filter direction

☐ Make this relationship active
☐ Assume referential integrity
☐ Apply security filter in both directions

OK Cancel

Creating Relationships

Step 3: In the second table drop-down list, choose the other table you want in the relationship and then select the other column you want to use, press **OK**

Create relationship

Select tables and columns that are related.

Products

Product_ID	Product_Name	Price
103	Laptop	400
104	Desktop	150
105	DVD Player	125

Customers

Customer_ID	Customer_Name	City
201	Maple Leaf Networking	Toronto
202	Ana Hu Printers	Beijing
203	Al Rashid Software Developers	Kuwait City

Cardinality

One to one (1:1)

Cross filter direction

Both

☒ Make this relationship active

☐ Assume referential integrity

☐ Apply security filter in both directions

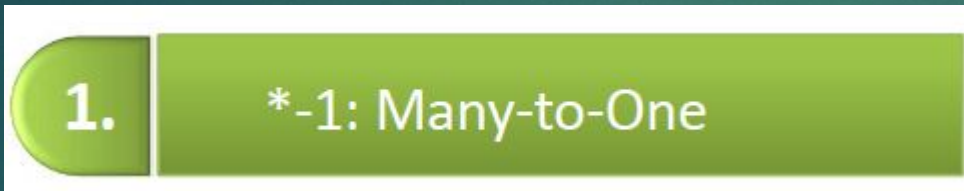
OK

Cancel



Cardinality

- While creating a relationship between two tables, you get two values that can be 1 or * on the two ends of the relationship among two tables, known as Cardinality of the Relationship
- There are four types of cardinality, as follows:



Cardinality

1. Many to one (*:1)

- A many-to-one relationship is a vital type of cardinality and default type of relationship
- In many to one relationship, the column in a given table can have more than one instance of the value, and the other related table is known as the lookup table and contained only one instance of a value

2. One to one (1:1)

- In a one-to-one (1:1) relationship, the column in one table has only one example of a specific value, and the different related table also contains only one instance of a specific value

Cardinality

3. One to many (1:*)

- In a one-to-many (1:*) relationship, the column in one table has only one instance of a specific value, and the other related table contains more than one instance of a value

4. Many to many (*:*)

- You can develop a many-to-many relationship between tables with composite models that removes the requirements for unique values in tables
- It also eliminates the previous workarounds, such as introducing new tables only to build relationships

Cross Filter Direction

- Each model relationship must be described with a cross filter direction
- Your selection decides the direction(s) that filters will propagate
- The possible cross filter options are dependent on the type of cardinality
- Single cross filter direction indicates **single direction**, and both show **both directions**
- A relationship that filters in both directions are commonly defined as bi-directional

Cross Filter Direction

Cardinality type	Cross filter options
One-to-many (or Many-to-one)	Single Both
One-to-one	Both
Many-to-many	Single (Table1 to Table2) Single (Table2 to Table1) Both



thank you